

# Chương 10: Giới thiệu về Mạng Nơ-ron Nhân tạo

Giảng viên

Đại học Đông Á

August 11, 2026

# Mục tiêu của Chương

- Khái niệm nền tảng về Nơ-ron sinh học và Nơ-ron nhân tạo.
- Kiến trúc Perceptron và Mạng Perceptron Đa lớp (MLP).
- Hiểu nguyên lý hoạt động của thuật toán Lan truyền ngược (Backpropagation).
- Triển khai mạng MLP cho bài toán Hồi quy và Phân loại.
- Sử dụng **Keras** và **TensorFlow** để xây dựng mô hình.
- Khám phá các API của Keras: Sequential, Functional, và Subclassing.
- Callbacks, TensorBoard và tinh chỉnh siêu tham số.

# Mục lục

1. Nơ-ron Sinh học và Khởi nguồn
2. Perceptron
3. Mạng Perceptron Đa lớp (MLP)
4. MLP Hồi quy & Phân loại
5. Triển khai MLP với Keras
6. Functional API & Subclassing API
7. Lưu Mô hình & Kỹ thuật nâng cao
8. Tinh chỉnh Siêu Tham số

# 1.1 Lịch sử Mạng Nơ-ron Nhân tạo (ANN)

- Ra đời năm 1943 bởi Warren McCulloch và Walter Pitts.
- Được truyền cảm hứng từ cấu trúc bộ não con người.
- Trải qua nhiều thập kỷ thăng trầm (AI Winter), mạng nơ-ron đã bùng nổ trở lại vào những năm 2010.
- Nguyên nhân bùng nổ: Dữ liệu lớn (Big Data), sức mạnh phần cứng (GPU), và các thuật toán huấn luyện cải tiến.

## 1.2 Nơ-ron Sinh học

- Tế bào thần kinh sinh học là một tế bào dị thường nằm ở vỏ não.
- Cấu trúc: Thân tế bào, sợi nhánh (Dendrites), và sợi trục (Axon).
- Tín hiệu được truyền qua các khớp thần kinh (Synapses).

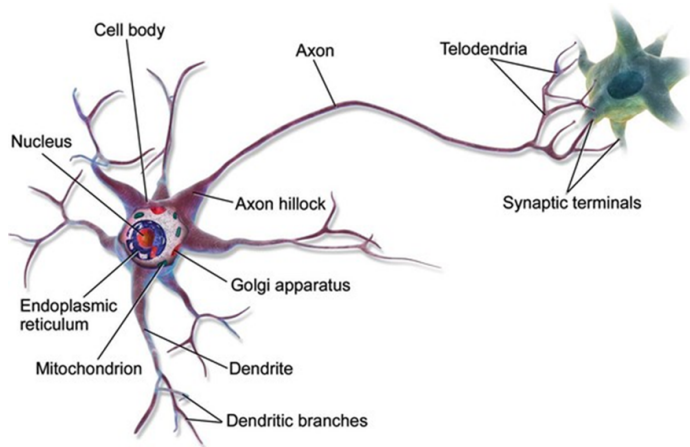


Figure: Hình 10-1: Cấu trúc của Nơ-ron sinh học

## 1.3 Từ Sinh học đến Mô hình Nhân tạo

- Mô hình nơ-ron nhân tạo đầu tiên (Mô hình McCulloch-Pitts) cực kỳ đơn giản.
- Một nơ-ron nhân tạo có một hoặc nhiều đầu vào (nhị phân), và một đầu ra.
- Nơ-ron kích hoạt (phát tín hiệu 1) khi một lượng đủ lớn các đầu vào của nó kích hoạt.
- Mặc dù đơn giản, nhưng khi kết nối chúng thành một mạng lưới, ta có thể tính toán bất kỳ biểu thức logic nào.

## 1.4 Các Phép toán Logic với Nơ-ron

- Mạng (a): Hàm IDENTITY.
- Mạng (b): Hàm AND (Chỉ sáng khi cả hai đều sáng).
- Mạng (c): Hàm OR (Sáng khi 1 trong 2 sáng).
- Mạng (d): Cổng logic phức tạp.

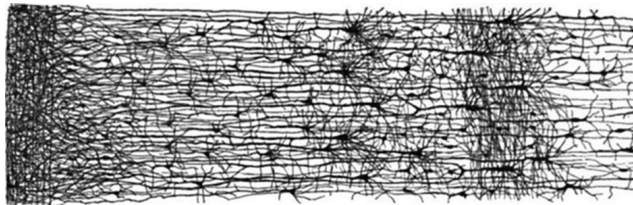


Figure: Hình 10-2: Các phép tính logic đơn giản

## 2.1 Khái niệm Perceptron

- Perceptron do Frank Rosenblatt phát minh năm 1957.
- Nó dựa trên một loại nơ-ron nhân tạo gọi là **TLU (Threshold Logic Unit)** hay **LTU (Linear Threshold Unit)**.
- Thay vì nhị phân, TLU nhận đầu vào là các con số.
- Mỗi kết nối đầu vào được gán một trọng số (Weight)  $w$ .

## 2.2 Hoạt động của TLU (Threshold Logic Unit)

- Tính tổng trọng số:  
$$z = w_1x_1 + w_2x_2 + \dots + b = \mathbf{w}^T \mathbf{x} + b.$$
- Sau đó áp dụng hàm bước (Step function) vào  $z$ :  
$$h_{\mathbf{w}}(\mathbf{x}) = \text{step}(z).$$
- Hàm bước thường dùng là hàm **Heaviside** (Trả về 1 nếu  $z \geq 0$ , ngược lại 0).

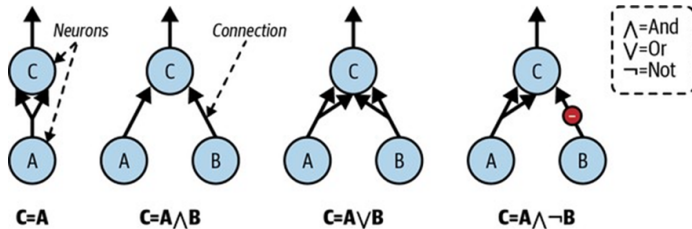


Figure: Hình 10-3: Threshold Logic Unit (TLU)

## 2.3 Kiến trúc Mạng Perceptron

- Perceptron bao gồm một lớp các TLU kết nối với tất cả các đầu vào (Lớp dày đặc - Dense Layer).
- Hình bên cho thấy một mạng Perceptron với 2 đầu vào, 1 nơ-ron bias, và 3 nơ-ron đầu ra.
- Mạng này có thể phân loại đa nhãn (Multi-output classification).

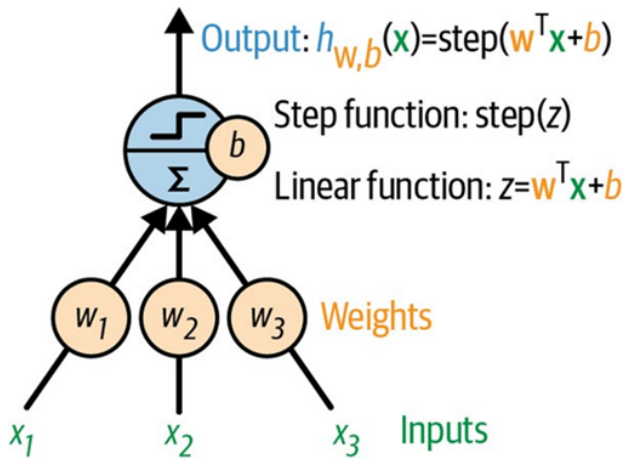


Figure: Hình 10-4: Mạng Perceptron

## 2.4 Điểm yếu của Perceptron: Bài toán XOR

- Năm 1969, Minsky và Papert chứng minh Perceptron có nhược điểm nghiêm trọng.
- Chúng là các bộ phân loại tuyến tính, do đó không thể giải bài toán **XOR** (**Exclusive OR**).
- Hình bên cho thấy bài toán XOR không thể bị cắt đôi bằng một đường thẳng.

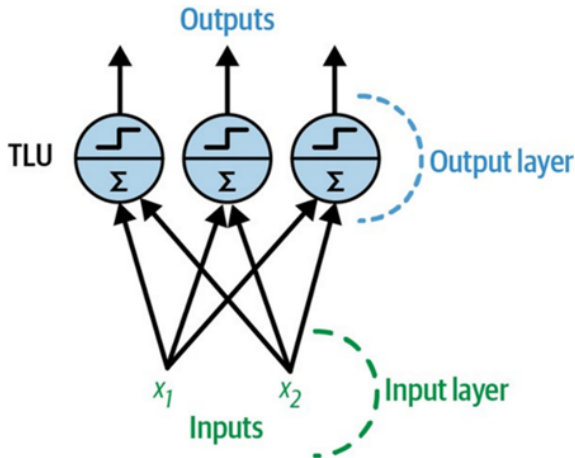


Figure: Hình 10-5: Điểm yếu giải quyết XOR

## 3.1 Mạng Perceptron Đa Lớp (MLP)

- Khắc phục sự cố XOR bằng cách xếp chồng nhiều Perceptron lên nhau → **Multi-Layer Perceptron (MLP)**.
- Kiến trúc MLP bao gồm:
  - ① Lớp Đầu vào (Input Layer).
  - ② Một hoặc nhiều Lớp Ẩn (Hidden Layers).
  - ③ Lớp Đầu ra (Output Layer).
- Bất kỳ mạng nơ-ron nào có từ 2 lớp ẩn trở lên được gọi là **Học Sâu (Deep Learning)**.

## 3.2 Kiến trúc MLP

- Mỗi lớp (ngoại trừ lớp ra) thường bao gồm một nơ-ron bias.
- Các nơ-ron ở một lớp được kết nối đầy đủ (Fully Connected) đến mọi nơ-ron ở lớp tiếp theo.
- Dữ liệu luôi trôi theo 1 chiều (từ trái qua phải), gọi là Mạng truyền thẳng (Feedforward Neural Network - FNN).

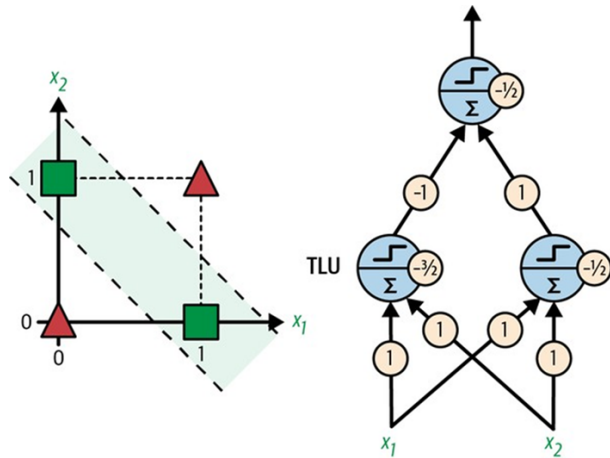


Figure: Hình 10-6: Kiến trúc Mạng Perceptron Đa lớp

## 3.3 Thuật toán Lan truyền ngược (Backpropagation)

- Năm 1986, Rumelhart, Hinton và Williams công bố bài báo giới thiệu thuật toán **Backpropagation** kết hợp **Gradient Descent**.
- Backprop là linh hồn của Deep Learning ngày nay.
- Quá trình hoạt động:
  - ➊ Lan truyền tiến (Forward Pass): Tính kết quả dự đoán.
  - ➋ Tính toán Lỗi (Compute Error): So sánh kết quả với thực tế.
  - ➌ Lan truyền ngược (Backward Pass): Tính đạo hàm (Gradient) của lỗi với từng trọng số (Sử dụng Chain Rule / Autodiff).
  - ➍ Cập nhật Trọng số (Gradient Descent Step).

## 3.4 Tại sao lại cần Hàm Kích hoạt (Activation Functions)?

- Để Backprop hoạt động, người ta thay hàm bước (Step function) bằng các hàm liên tục có đạo hàm khác không.
- Nếu chỉ cộng dồn tuyến tính, mạng nơ-ron 100 lớp cũng chỉ ngang bằng mạng 1 lớp (Vì hàm tuyến tính tổ hợp lại vẫn là tuyến tính).
- Hàm kích hoạt mang tính **phi tuyến tính (Non-linearity)**, giúp mạng học được các hàm phức tạp.

## 3.5 Các Hàm Kích Hoạt Phổ Biến

- **Sigmoid:** Ép đầu ra về khoảng  $[0, 1]$ . Dùng cho phân loại nhị phân.
- **Tanh:** Ép về  $[-1, 1]$ , có tâm tại 0.
- **ReLU:** Bằng 0 nếu âm, giữ nguyên nếu dương. Nhanh, đạo hàm tốt, là mặc định cho các lớp ẩn hiện nay.

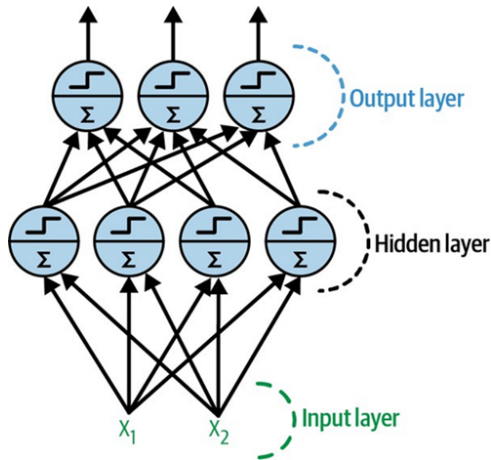


Figure: Hình 10-7: Bảng các Hàm kích hoạt (Activation Functions)

## 4.1 MLP Hồi quy (Regression MLP)

- Hồi quy dự đoán một giá trị liên tục (Ví dụ: Giá nhà).
- Lớp đầu ra chỉ cần **1 nơ-ron duy nhất** (Không dùng hàm kích hoạt).
- Nếu dự đoán giá trị phải luôn dương, dùng hàm kích hoạt **ReLU** hoặc **Softplus** ở đầu ra.
- Hàm mất mát (Loss function): **MSE (Mean Squared Error)** hoặc **MAE (Mean Absolute Error)**.

## 4.2 Cấu hình chuẩn cho MLP Hồi quy

| Hyperparameter            | Typical Value                                           |
|---------------------------|---------------------------------------------------------|
| Nơ-ron đầu vào (Input)    | Một nơ-ron cho mỗi đặc trưng (Feature).                 |
| Số lớp ẩn (Hidden Layers) | Tùy thuộc bài toán, thường từ 1 đến 5.                  |
| Nơ-ron mỗi lớp ẩn         | Thường từ 10 đến 100.                                   |
| Nơ-ron đầu ra (Output)    | 1 (Nếu tiên đoán 1 giá trị), N (Nếu tiên đoán N chiều). |
| Hàm kích hoạt (Lớp ẩn)    | ReLU (Hoặc SELU, Mish).                                 |
| Hàm kích hoạt (Đầu ra)    | None (Liên tục), ReLU (Dương), Sigmoid (Khoảng 0-1).    |
| Hàm mất mát (Loss)        | MSE, MAE, hoặc Huber Loss.                              |

Table: Hình 10-8: Kiến trúc MLP Hồi quy chuẩn

## 4.3 MLP Phân loại (Classification MLP)

- **Phân loại nhị phân (Binary):** Dùng 1 nơ-ron đầu ra với hàm **Sigmoid**. (Xác suất từ 0 đến 1). Loss: Binary Crossentropy.
- **Phân loại đa nhãn (Multilabel):** Vd nhận dạng email vừa là spam vừa là urgent. Dùng N nơ-ron đầu ra, mỗi nơ-ron có hàm **Sigmoid**. Loss: Binary Crossentropy.
- **Phân loại đa lớp (Multiclass):** Vd nhận diện MNIST (Từ 0 đến 9). Dùng N nơ-ron đầu ra với hàm **Softmax**. Loss: Categorical Crossentropy.

## 4.4 Cấu hình chuẩn cho MLP Phân loại

| Hyperparameter    | Binary              | Multilabel                        | Multiclass                  |
|-------------------|---------------------|-----------------------------------|-----------------------------|
| Input & Hidden    |                     | Giống như Hồi quy (1-5 lớp, ReLU) |                             |
| Output Neurons    | 1                   | 1 cho mỗi nhãn                    | 1 cho mỗi lớp (Tổng cộng N) |
| Output Activation | <b>Sigmoid</b>      | <b>Sigmoid</b>                    | <b>Softmax</b>              |
| Loss Function     | Binary Crossentropy | Binary Crossentropy               | Categorical Crossentropy    |

Table: Hình 10-9: Kiến trúc MLP Phân loại chuẩn

## 5.1 TensorFlow và Keras

- **TensorFlow:** Thư viện mã nguồn mở cho Học sâu của Google. Xử lý đồ thị tính toán mạnh mẽ trên CPU, GPU, TPU.
- **Keras:** API bậc cao (High-level API) của TensorFlow. Cực kỳ thân thiện với người dùng, dễ học, nhưng đủ linh hoạt để tùy chỉnh mô hình chuyên sâu.
- Hướng dẫn sau sử dụng bộ dữ liệu **Fashion MNIST** (Nhận diện ảnh 28x28 của các mặt hàng quần áo thời trang).

## 5.2 Sequential API: Xây dựng mô hình

- Keras Sequential API tạo mô hình bằng cách xếp chồng các lớp (Layer) tuần tự.

```
import tensorflow as tf
from tensorflow import keras

model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu"),
    keras.layers.Dense(100, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
])
model.summary() # Xem tóm tắt cấu trúc mô hình
```

## 5.3 Giải thích Mô hình Sequential

- Flatten: Lớp chuyển đổi ảnh ma trận 2D (28x28) thành mảng 1D (784). (Không có trọng số).
- Dense(300): Lớp ẩn đầu tiên với 300 nơ-ron kết nối đầy đủ. Số tham số =  $784 \times 300 + 300$  (bias) = 235,500.
- Dense(100): Lớp ẩn thứ hai với 100 nơ-ron. (30,100 tham số).
- Dense(10): Lớp đầu ra với 10 nơ-ron (cho 10 lớp mặt hàng). Dùng Softmax để xuất ra xác suất tổng bằng 1.

## 5.4 Biên dịch (Compile) và Huấn luyện (Fit)

```
# Bước 1: Compile - Khai báo Loss và Optimizer
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="sgd",
              metrics=["accuracy"])

# Bước 2: Fit - Huấn luyện mô hình (Lan truyền ngược)
history = model.fit(X_train, y_train, epochs=30,
                   validation_data=(X_valid, y_valid))
```

- `sparse_categorical_crossentropy`: Dùng vì nhãn Y là các số nguyên rời rạc (0-9).
- `sgd`: Stochastic Gradient Descent.
- `epochs=30`: Huấn luyện duyệt qua toàn bộ tập dữ liệu 30 lần.

## 5.5 Đường cong Học tập (Learning Curves)

- Object history chứa dữ liệu loss và accuracy của train/valid sau mỗi epoch.
- Nhìn vào biểu đồ, Loss liên tục giảm dần (tốt).
- Accuracy liên tục tăng (tốt).
- Nếu Validation Loss đi ngang hoặc tăng lên, mô hình đang bị **Overfitting**.

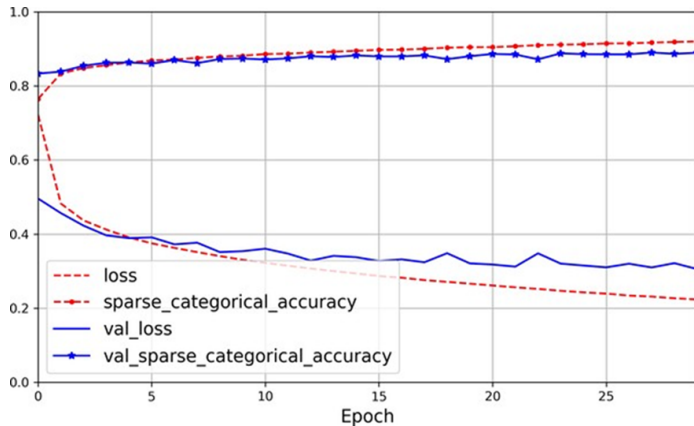


Figure: Hình 10-11: Đồ thị Loss và Accuracy theo số vòng lặp (Epoch)

## 6.1 Vượt giới hạn với Functional API

- Mô hình tuần tự (Sequential) rất phổ biến, nhưng đôi khi ta cần cấu trúc phi tuần tự.
- Ví dụ: **Wide & Deep Neural Network**.
- Kiến trúc này nối trực tiếp lớp Đầu vào (Input) với lớp Đầu ra (Output) song song cùng với Lớp ẩn sâu.
- Cho phép mạng học được cả **Mẫu sâu (Deep patterns)** lẫn **Quy luật nông (Simple rules)**.

## 6.2 Kiến trúc Wide & Deep

- Các mô hình MLP thông thường ép mọi dữ liệu chảy qua một chuỗi các lớp ẩn, khiến các quy luật đơn giản ban đầu có thể bị biến dạng hoặc "quên" mất.
- Kiến trúc Wide & Deep chia nhánh để bảo toàn các đặc trưng đầu vào gốc.

Ankle boot



Pullover



Trouser



Figure: Hình 10-12: Mạng Nơ-ron Rộng và Sâu (Wide & Deep)

## 6.3 Code: Functional API

```
input_ = keras.layers.Input(shape=X_train.shape[1:])
hidden1 = keras.layers.Dense(30, activation="relu")(input_)
hidden2 = keras.layers.Dense(30, activation="relu")(hidden1)

# Ghep noi (Concatenate) duong "Wide" va "Deep"
concat = keras.layers.Concatenate()([input_, hidden2])

output = keras.layers.Dense(1)(concat)
model = keras.models.Model(inputs=[input_], outputs=[output])
```

- Trong Functional API, ta phải gọi từng lớp như một hàm và truyền input vào.
- Cuối cùng, bọc tất cả lại trong `keras.models.Model`.

## 6.4 Functional API: Nhiều luồng dữ liệu (Multiple Inputs)

- Sẽ ra sao nếu ta muốn gửi một nhóm đặc trưng vào "Đường rộng", và một nhóm đặc trưng khác vào "Đường sâu"?
- Keras Functional API có thể tạo nhiều luồng Input.
- Lúc gọi hàm `fit()`, ta sẽ truyền (`X_train_A`, `X_train_B`).

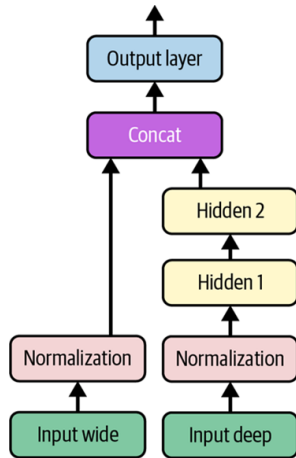


Figure: Hình 10-14: Xử lý nhiều luồng đầu vào

## 6.5 Functional API: Nhiều đầu ra (Multiple Outputs)

- Tương tự, một mạng cũng có thể có nhiều đầu ra.
- Ví dụ: Phát hiện vật thể trên ảnh (1 Output là Phân loại vật, 1 Output khác là Tọa độ đóng khung vật).
- Mỗi Output sẽ có một loss riêng biệt. Keras tính toán theo tổng các loss có hệ số nhân.

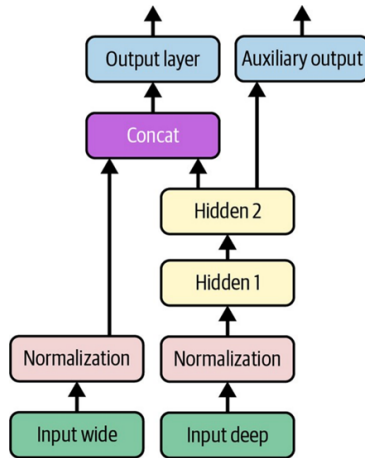


Figure: Hình 10-15: Mạng có nhiều đầu ra

## 6.6 Subclassing API (Lập trình Hướng đối tượng)

- Cả Sequential và Functional API đều là "Tuyên bố" (Declarative). Nghĩa là mô hình là đồ thị tĩnh.
- Nếu bạn cần một mô hình với kiến trúc "Động", có vòng lặp, có if/else hoạt động linh hoạt, hãy dùng **Subclassing API**.

```
class WideAndDeepModel(keras.models.Model):  
    def __init__(self, units=30, activation="relu", **kwargs):  
        super().__init__(**kwargs)  
        self.hidden1 = keras.layers.Dense(units, activation=activation)  
        self.hidden2 = keras.layers.Dense(units, activation=activation)  
        self.main_output = keras.layers.Dense(1)  
  
    def call(self, inputs): # Định nghĩa cách truyền tiền tại đây  
        hidden1 = self.hidden1(inputs)  
        hidden2 = self.hidden2(hidden1)  
        concat = keras.layers.concatenate([inputs, hidden2])  
        return self.main_output(concat)
```

## 7.1 Lưu và Khôi phục Mô hình

- Quá trình huấn luyện Mạng Nơ-ron tốn rất nhiều thời gian (Có thể vài ngày). Bạn cần lưu lại!
- Keras lưu mô hình theo định dạng HDF5 (.h5). Nó chứa toàn bộ Trọng số, Cấu trúc, Optimizer.

```
# Lưu model
model.save("my_keras_model.h5")

# Phục hồi model
model = keras.models.load_model("my_keras_model.h5")
```

## 7.2 Callbacks: Trợ lý huấn luyện

- Keras cho phép gọi các hàm **Callback** ở mỗi cuối Epoch hoặc cuối Batch.
- **ModelCheckpoint**: Tự động lưu mô hình mỗi khi nó "Đạt điểm cao nhất" trên tập Validation (Tránh mất mô hình tốt nhất nếu bị sập nguồn).

```
checkpoint_cb = keras.callbacks.ModelCheckpoint("my_best_model.h5",  
                                              save_best_only=True)  
history = model.fit(X_train, y_train, epochs=10,  
                  validation_data=(X_valid, y_valid),  
                  callbacks=[checkpoint_cb])
```

## 7.3 Dừng sớm (Early Stopping)

- Không ai biết cần đặt epochs bao nhiêu là vừa. Đặt lớn quá sẽ gây Overfitting.
- **EarlyStopping**: Tự động "Rút phích cắm" dừng huấn luyện nếu nhận thấy Loss trên tập Validation không còn cải thiện.

```
early_stopping_cb = keras.callbacks.EarlyStopping(patience=10,  
                                                    restore_best_weights=True)  
# patience=10 nghĩa là đợi thêm 10 epoch xem có hy vọng không.  
history = model.fit(X_train, y_train, epochs=1000, # Dạy epoch 1000 cho thoải mái  
                    validation_data=(X_valid, y_valid),  
                    callbacks=[checkpoint_cb, early_stopping_cb])
```

## 7.4 TensorBoard: Công cụ Giám sát tối thượng

- TensorFlow cung cấp **TensorBoard**, một giao diện đồ họa siêu cấp để theo dõi quá trình huấn luyện trực tiếp.
- Nó vẽ đồ thị Loss, Accuracy theo thời gian thực.
- Hiển thị sơ đồ đồ thị kết nối Lớp (Graph).
- Cực kỳ hữu ích để gỡ lỗi các mạng khổng lồ.
- Ghi log bằng cách truyền `keras.callbacks.TensorBoard(log_dir="logs")` vào `callbacks`.

## 8.1 Tinh chỉnh Siêu Tham số (Hyperparameters)

- Mạng Nơ-ron có độ linh hoạt tuyệt đối. Đồng nghĩa với việc có hàng tá thông số cần tinh chỉnh!
- Làm sao để biết mạng nên có 2 lớp hay 100 lớp ẩn?
- Phương pháp cơ bản: Sử dụng GridSearchCV hoặc RandomizedSearchCV.
- Tuy nhiên, với Mạng Sâu, RandomizedSearch hiệu quả hơn nhiều. Bạn cũng có thể dùng công cụ **Keras Tuner**.

## 8.2 Số lượng Lớp Ẩn (Hidden Layers)

- Mạng có 1 lớp ẩn với cấu trúc rộng theo lý thuyết có thể xấp xỉ bất kỳ hàm nào. Nhưng rất tốn tham số!
- **Mạng Sâu (Nhiều lớp ẩn):** Có hiệu suất tham số lớn hơn (parameter efficiency). Nó học cách phân cấp dữ liệu.
- Ví dụ nhận diện mặt: Lớp 1 học vẽ đoạn thẳng, lớp 2 vẽ thành mắt/mũi/miệng, lớp 3 gộp thành khuôn mặt.
- Kinh nghiệm: Hãy bắt đầu với 1 hoặc 2 lớp ẩn (Nó sẽ hoạt động tốt cho đa số bài toán). Đối với ảnh phức tạp, cần hàng chục lớp.

## 8.3 Số lượng Nơ-ron trên mỗi Lớp Ẩn

- Cách thiết kế cổ điển (Hình phễu): Lớp đầu có nhiều nơ-ron (Vd: 300), lớp tiếp theo giảm dần (100), rồi (50)... nhằm tinh lọc dữ liệu.
- Tuy nhiên, nghiên cứu chỉ ra rằng **đặt số nơ-ron bằng nhau** ở mọi lớp ẩn (ví dụ: tất cả 150) cho ra kết quả tương tự, thậm chí tốt hơn, mà lại giảm bớt số tham số bạn phải nghĩ cách cấu hình.
- Mẹo vặt chiến thuật: Hãy chọn mô hình có số lớp và số nơ-ron "thừa thãi" nhiều hơn mức cần thiết, sau đó sử dụng **Early Stopping** và **Dropout** để ngăn nó Overfitting.

## 8.4 Tốc độ học (Learning Rate)

- Tốc độ học ( $\eta$ ) là siêu tham số **QUAN TRỌNG NHẤT**. Nếu chỉ có thời gian tinh chỉnh 1 tham số, hãy tinh chỉnh cái này!
- Nếu  $\eta$  quá lớn: Mạng sẽ nhảy qua lại không thể hội tụ (Diverge).
- Nếu  $\eta$  quá nhỏ: Mất rất nhiều ngày tháng để huấn luyện.
- Giá trị khởi điểm kinh điển:  $10^{-3}$  hoặc  $10^{-4}$ .

## 8.5 Kích thước Lô (Batch Size) & Optimizer

- **Batch Size:** Thường từ 32 đến 128. Batch Size quá to sẽ huấn luyện nhanh nhờ GPU nhưng có nguy cơ rơi vào tối ưu cục bộ. Batch Size nhỏ tốn CPU, tốn thời gian nhưng mang tính "nhảy múa" cao, dễ thoát hố sâu cục bộ.
- **Optimizer:** Thuật toán **Adam** Optimizer (Sẽ học ở chương 11) được chứng minh tốt hơn nhiều so với **SGD** cổ điển và đang được dùng mặc định ở hầu hết bài toán Học sâu.

# HỎI & ĐÁP